CHAPTER 1 · SOFTWARE ENGINEERING

Introduction to Software Engineering

The 10 end-of-chapter exercises, explained in simple words with real examples

Based on Ian Sommerville, Software Engineering, 10th Edition

KEY IDEAS FIRST

Three definitions you need for every answer

Software

Programs **plus** documentation **plus** configuration data. Not just code.

Software engineering

An engineering discipline that covers **all aspects** of software production — from first idea to maintenance.

Two kinds of product

Generic: sold to anyone (e.g. Excel).
Custom: built for one customer (e.g. a hospital system).

Simple analogy: a house is not just walls — it also comes with plans, wiring diagrams and a warranty.

KEY IDEAS FIRST

Every software process has four activities

1

Specification

Decide **what** to build and its limits

2

Development

Design and **program** it

3

Validation

Check it does what the customer wants

4

Evolution

Change it as needs change

Example: a food-delivery app — list the features, code them, test them with real users, then keep updating after launch.

KEY IDEAS FIRST

What makes software "good"? Four attributes

Maintainability

Easy to change when needs change.

Dependability & security

Reliable, safe, and protected from attackers.

Efficiency

Doesn't waste memory, CPU or battery.

Acceptability

Users understand it, like it, and it fits their other systems.

PART 2

The 10 exercises

For each one: the question, the key points of the answer, and a one-line summary.

EXERCISE 1.1

Why is professional software for a customer more than just the programs?

Documentation

User manuals, requirements and design documents so people can use and understand it.

Configuration data

Settings, setup files and data the programs need in order to run.

Long life

It will be maintained for years, often by other people who need clear information.

IN SHORT The customer pays for a complete, usable and maintainable system — not just code.

EXERCISE 1.2

What is the main difference between generic and custom software? What does it mean for users of generic products?

Generic product

The **developer** owns the specification. Example: Microsoft Word, Photoshop.

Custom product

The **customer** owns the specification. Example: a system built for one hospital.

Meaning for users

No control over features or changes. Users must adapt their work to the software.

IN SHORT The key difference is who decides what the software does: the developer or the customer.

EXERCISE 1.3

Why is it usually cheaper in the long run to use software engineering methods?

Most cost comes later

For most systems, the biggest cost is **changing** the software after it is in use.

Bugs found early

A mistake fixed during requirements is far cheaper than one fixed after release.

Easy to change

Good design, documentation and tests make future changes faster and safer.

IN SHORT Engineering takes more effort at the start but saves much more during testing and maintenance.

EXERCISE 1.4

Give examples of ethical issues that affect software engineering.

Confidentiality

Keep employer and client information secret, e.g. not sharing customer data.

Competence

Don't accept work beyond your skills, e.g. a safety system you can't build.

Intellectual property

Respect copyrights, patents and licences — don't copy code illegally.

Misuse & privacy

Don't hack, spread viruses, or collect user data without permission.

IN SHORT Engineers must act honestly and responsibly, not only solve technical problems.

EXERCISE 1.5

Why do different application types need specialized techniques?

Application type	Example	Special technique focus
Embedded control	Car brakes (ABS)	Real-time response, safety
Interactive / web	Online banking, shop	Security, many users at once
Batch processing	Monthly payroll, billing	Huge data, efficiency
Entertainment	Video games	User experience, speed
Modeling & simulation	Weather forecast	Accuracy, heavy computation

IN SHORT Each type has different priorities, so no single method fits all systems.

EXERCISE 1.6

Why are process, dependability, requirements and reuse relevant to ALL software?

Process

Every system needs a planned, managed process — even a small mobile app.

Dependability & performance

Every user expects software to work correctly, safely and fast enough.

Requirements

You must know what users need before building, or you build the wrong thing.

Reuse

Using existing libraries and components saves time and money for any system.

IN SHORT These are the foundations; only the techniques on top of them change.

EXERCISE 1.7

How can electronic connectivity between teams support software engineering?

Communication

Chat, email and video calls connect teams across cities and time zones.

Shared code & tools

Git/GitHub, issue trackers and shared docs keep everyone on one version.

Automation

Cloud CI/CD builds and tests every change automatically.

Reuse & knowledge

Online libraries, services and documentation are available to all.

IN SHORT Connected teams can work on one system as if they were in the same room.

EXERCISE 1.8

Non-certified people can still practise software engineering. What are the drawbacks?

Lower quality

May not know testing, design or security methods, so more bugs.

No accountability

No licence to lose and no required code of ethics.

Hard to judge

Clients can't easily tell a skilled engineer from an unskilled one.

Safety risk

Dangerous in critical systems: medical, cars, banking.

Poor reputation

Bad projects damage trust in the whole profession.

Legal issues

May ignore laws on privacy, licensing and contracts.

EXERCISE 1.9

Give an example for each of the eight ACM/IEEE Code of Ethics principles.

1. Public

Report a safety bug in a medical device, even if it delays release.

2. Client & employer

Don't use company code or data for your own side project.

3. Product

Don't ship code you know is buggy just to meet a date.

4. Judgment

Refuse to sign a test report you didn't really check.

5. Management

Managers set realistic deadlines and fair estimates.

6. Profession

Be honest with clients about what the system can't do.

7. Colleagues

Credit teammates' work and help juniors learn.

8. Self

Keep learning new tools, methods and security practices.

EXERCISE 1.10

What societal challenges come with building drones?

Privacy
Cameras can watch people and homes without permission.

Safety
Software failure can crash a drone onto people or cars.

Security
Hackers could hijack or jam a drone's control.

Responsibility
Who is to blame in an accident: the pilot, the maker, or the software?

Misuse
Weapons, smuggling, or flying near airports.

Laws & jobs
Rules lag behind the technology; some jobs may disappear.

SUMMARY

Six ideas that answer most of Chapter 1

1. Software = programs + documents + data
2. Generic vs custom: who owns the spec
3. Most cost is after delivery — engineer early
4. Different systems need different techniques
5. Fundamentals apply to every system
6. Engineers have ethical duties to society

QUICK CHECK

Questions?

Try it: pick an app you use every day. Is it generic or custom? Which application type is it? Which ethical issue could affect it?